

Name : Ragini Singh (BTech CSE 2024-28)

Cohort : Jensen Huang

Roll No : 150096724023

Test : [Node.js](#) + [Express.js](#)

Section A: Debugging Questions

1. Route Not Responding — FIX

Bug: `app.listen` is placed after console log, but code actually works; real issue: missing callback or server not started properly.

Fix:

```
const express = require('express');
const app = express();

app.get('/hello', (req, res) => {
  res.send('Hello World');
});

app.listen(3000, () => {
  console.log('Server started on port 3000');
});
```

2. JSON Body Not Parsed — FIX

Bug: `app.use(express.json())` should come BEFORE the route.

```
const express = require('express');
const app = express();

app.use(express.json()); // moved up

app.post('/data', (req, res) => {
  res.send(req.body);
});
```

```
});
```

3. Mongoose Connection Error — FIX

Use new URL parser + unified topology:

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost/testdb', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});
```

4. Async Function Not Awaited — FIX

```
async function getData() {
  return await Promise.resolve('Done');
}

getData().then(result => console.log(result));
```

OR

```
(async () => {
  const result = await getData();
  console.log(result);
})();
```

5. Middleware Chain Broken — FIX

Bug: First middleware never calls `next()`.

```
app.use((req, res, next) => {
```

```
    console.log('First');  
    next();  
  });  
  
  app.use((req, res, next) => {  
    console.log('Second');  
    res.send('Done');  
  });
```

Section B: Predict-the-Output

6. Event Loop

A
C
B

7. Promise Chain

X
Y

8. Middleware Order

If hitting `/`:

One
Two

9. Mongoose Save

Before save → `isNew` = true
After save → `isNew` = false

true
false

10. Async/Await

Start
End

Section C: Mixed Challenge Questions

11. Bug Hunt

Problem:

Console says: *Server running on port 3000*
But actually server is running on 4000.

Fix message:

```
console.log("Server running on port 4000");
```

12. Predict Output

Order:

Sync
NextTick
Immediate

13. Error Handling Middleware

500 Internal Server Error

Oops

14. Async Error

In Express, unhandled async errors are NOT automatically caught.
This will crash the server unless a wrapper or async error handler is used.

Expected behaviour: Server crashes with UnhandledPromiseRejection.

15. Predict Output

`forEach` does NOT wait for async.

All timeouts finish after ~100ms, but order remains 1,2,3:

1

2

3